GTSRB Traffic Sign Recognition

Date: 26/01/2026

Built by:

- Yasser Hegazy: 120210916
- Khaled Abu Kwaik: 120210414
- Abdullah Abu Zaid: 120211559

Overview

Complete analysis and modeling of German Traffic Sign Recognition Benchmark (GTSRB) dataset.

- **Resolution**: 32x32 (native GTSRB)
- **Models**: Custom CNN and EfficientNetB0 (both optimized for 32x32)

Step 0: Import Libraries and Configuration

*[1]:
```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from PIL import Image
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
import tensorflow as tf
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Dense, Flatten, Dropout, BatchNormalization, Input, Lambda, GlobalAveragePooling2D
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras.applications.efficientnet import preprocess_input as preprocess_efficientnet

# Set Dataset Path
DATASET_PATH = r"D:\Downloads folder\Compressed\archive"
TRAIN_PATH = os.path.join(DATASET_PATH, 'Train')
TEST_PATH = os.path.join(DATASET_PATH, 'Test')

print(f"Dataset Path: {DATASET_PATH}")

# Define image constants
IMG_HEIGHT = 32
IMG_WIDTH = 32
CHANNELS = 3
```

```
Dataset Path: D:\Downloads folder\Compressed\archive
```

Step 1: Load and Explore Dataset

[2]:
```
# Load CSV files
train_df = pd.read_csv(os.path.join(DATASET_PATH, 'Train.csv'))
test_df = pd.read_csv(os.path.join(DATASET_PATH, 'Test.csv'))
meta_df = pd.read_csv(os.path.join(DATASET_PATH, 'Meta.csv'))

print(f"Training samples: {len(train_df)}")
print(f"Test samples: {len(test_df)}")
print(f"Meta (Classes) info: {len(meta_df)}")

# Verify number of classes
num_classes = len(train_df['ClassId'].unique())
print(f"Number of classes: {num_classes}")
assert num_classes == 43, "Expected 43 classes!"

# Display first few rows
train_df.head()
```

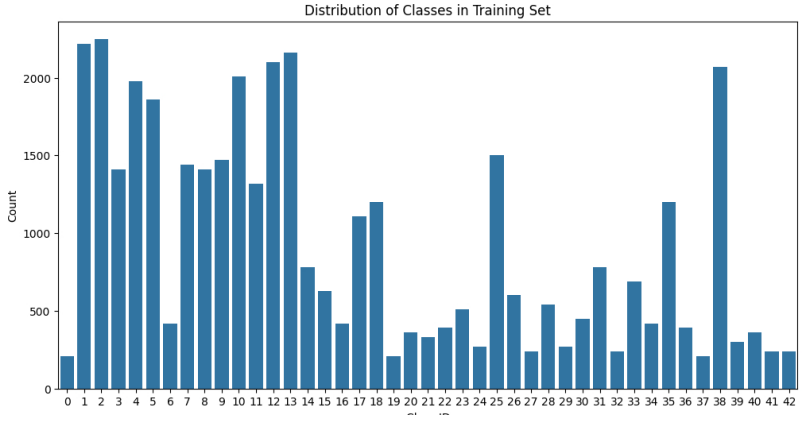
```
Training samples: 39209
Test samples: 12630
Meta (Classes) info: 43
Number of classes: 43
```

[2]:

	Width	Height	Roi.X1	Roi.Y1	Roi.X2	Roi.Y2	ClassId	Path
0	27	26	5	5	22	20	20	Train/20/00020_00000_00000.png
1	28	27	5	6	23	22	20	Train/20/00020_00000_00001.png
2	29	26	6	5	24	21	20	Train/20/00020_00000_00002.png
3	28	27	5	6	23	22	20	Train/20/00020_00000_00003.png
4	28	26	5	5	23	21	20	Train/20/00020_00000_00004.png

Step 2: Exploratory Data Analysis

[3]:
```
# Class Distribution
plt.figure(figsize=(12, 6))
sns.countplot(x='ClassId', data=train_df)
plt.title('Distribution of Classes in Training Set')
plt.xlabel('Class ID')
plt.ylabel('Count')
plt.show()
```



Class ID

[4]:
```
# Visualize Sample Images
def show_samples(df, path_col='Path', num_samples=5):
    plt.figure(figsize=(15, 3))
    samples = df.sample(num_samples)
    for i, (_, row) in enumerate(samples.iterrows()):
        img_path = os.path.join(DATASET_PATH, row[path_col])
        img = Image.open(img_path)
        plt.subplot(1, num_samples, i+1)
        plt.imshow(img)
        plt.title(f"Class: {row['ClassId']}")
        plt.axis('off')
    plt.show()

show_samples(train_df)
```

Step 3: Data Preprocessing and Loading (32x32)

[5]:
```
def load_and_preprocess_image(path, target_size=(32, 32)):
    """Load image and resize to target dimensions"""
    try:
        image = Image.open(path)
        image = image.resize(target_size)
        image = np.array(image)
        return image
    except Exception as e:
        print(f"Error loading {path}: {e}")
        return None

# Load all training images (32x32 - NATIVE GTSRB)
data = []
labels = []

print("Loading training images (32x32)...")
for index, row in train_df.iterrows():
    img_path = os.path.join(DATASET_PATH, row['Path'])
    img = load_and_preprocess_image(img_path, target_size=(IMG_WIDTH, IMG_HEIGHT))
    if img is not None:
        data.append(img)
        labels.append(row['ClassId'])
    if index % 5000 == 0:
        print(f"Processed {index} images")

print(f"Loaded {len(data)} images (32x32)")
```

```
Loading training images (32x32)...
Processed 0 images
Processed 5000 images
Processed 10000 images
Processed 15000 images
Processed 20000 images
Processed 25000 images
Processed 30000 images
Processed 35000 images
Loaded 39209 images (32x32)
```

[6]:
```
# Prepare data for both models at 32x32
raw_data = np.array(data)  # Keep raw data [0-255]
data_normalized = raw_data.astype('float32') / 255.0  # Normalize [0-1] for CNN

# One-hot encode labels
labels = to_categorical(labels, num_classes)

print(f"Raw data shape (32x32): {raw_data.shape}")
print(f"Normalized data shape (32x32): {data_normalized.shape}")
print(f"Labels shape: {labels.shape}")
```

```
Raw data shape (32x32): (39209, 32, 32, 3)
Normalized data shape (32x32): (39209, 32, 32, 3)
Labels shape: (39209, 43)
```

[7]:
```
# Split data for both models (both use 32x32)
# CNN: normalized [0-1]
X_train_cnn, X_val_cnn, y_train, y_val = train_test_split(data_normalized, labels, test_size=0.2, random_state=42)
print(f"CNN Training set (32x32): {X_train_cnn.shape}")
print(f"CNN Validation set (32x32): {X_val_cnn.shape}")

# EfficientNetB0: raw data with specific preprocessing
X_train_eff, X_val_eff, y_train_eff, y_val_eff = train_test_split(raw_data, labels, test_size=0.2, random_state=42)
X_train_eff = preprocess_efficientnet(X_train_eff.astype('float32'))
X_val_eff = preprocess_efficientnet(X_val_eff.astype('float32'))
print(f"\nEfficientNetB0 Training set (32x32): {X_train_eff.shape}")
print(f"EfficientNetB0 Validation set (32x32): {X_val_eff.shape}")
```

```
CNN Training set (32x32): (31367, 32, 32, 3)
CNN Validation set (32x32): (7842, 32, 32, 3)

EfficientNetB0 Training set (32x32): (31367, 32, 32, 3)
EfficientNetB0 Validation set (32x32): (7842, 32, 32, 3)
```

Step 4: Data Augmentation

*[8]:
```
# Data augmentation
datagen = ImageDataGenerator(
    rotation_range=10,
    zoom_range=0.1,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=False  # Traffic signs are direction sensitive
)

# Fit on training data
datagen.fit(X_train_cnn)

# Visualize augmented images
x_batch, y_batch = next(datagen.flow(X_train_cnn[:9], y_train[:9], batch_size=9))
plt.figure(figsize=(10, 10))
for i in range(9):
    plt.subplot(3, 3, i+1)
    plt.imshow(x_batch[i])
    plt.axis('off')
plt.suptitle("Augmented Images (32x32)")
plt.show()
```

Augmented Images (32x32)

Step 5: Build and Train Custom CNN (32x32)

[19]:
```
def build_cnn_model():
    """Build a custom CNN model for traffic sign classification (32x32)"""
    model = Sequential([
        Conv2D(32, (3, 3), activation='relu', input_shape=(IMG_HEIGHT, IMG_WIDTH, CHANNELS)),
        BatchNormalization(),
        MaxPooling2D(pool_size=(2, 2)),

        Conv2D(64, (3, 3), activation='relu'),
        BatchNormalization(),
        MaxPooling2D(pool_size=(2, 2)),

        Conv2D(128, (3, 3), activation='relu'),
        BatchNormalization(),
        MaxPooling2D(pool_size=(2, 2)),

        Flatten(),
        Dense(256, activation='relu'),
        Dropout(0.5),
        Dense(num_classes, activation='softmax')
    ])
    return model

# Build and compile Custom CNN
model_cnn = build_cnn_model()
model_cnn.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
print("Custom CNN Model (32x32):")
model_cnn.summary()
```

```
Custom CNN Model (32x32):
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 30, 30, 32)	896
batch_normalization_3 (BatchNormalization)	(None, 30, 30, 32)	128
max_pooling2d_3 (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_4 (Conv2D)	(None, 13, 13, 64)	18,496
batch_normalization_4 (BatchNormalization)	(None, 13, 13, 64)	256
max_pooling2d_4 (MaxPooling2D)	(None, 6, 6, 64)	0
conv2d_5 (Conv2D)	(None, 4, 4, 128)	73,856
batch_normalization_5 (BatchNormalization)	(None, 4, 4, 128)	512
max_pooling2d_5 (MaxPooling2D)	(None, 2, 2, 128)	0
flatten_1 (Flatten)	(None, 512)	0
dense_4 (Dense)	(None, 256)	131,328
dropout_2 (Dropout)	(None, 256)	0
dense_5 (Dense)	(None, 43)	11,051

Total params: 236,523 (923.92 KB)

Trainable params: 236,075 (922.17 KB)

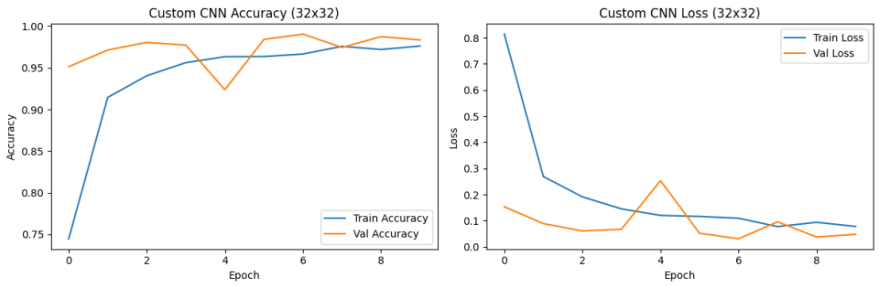
Non-trainable params: 448 (1.75 KB)

[21]:
```
# Train Custom CNN (32x32)
print("\nTraining Custom CNN (32x32)...")
history_cnn = model_cnn.fit(
    datagen.flow(X_train_cnn, y_train, batch_size=32),
    validation_data=(X_val_cnn, y_val),
    epochs=10,
    verbose=1
)
print("Custom CNN training complete!")
```

```
Training Custom CNN (32x32)...
Epoch 1/10
981/981 ━━━━━━━━━━━━━━━━━━━━ 18s 18ms/step - accuracy: 0.7447 - loss: 0.8140 - val_accuracy: 0.9513 - val_loss: 0.1532
Epoch 2/10
981/981 ━━━━━━━━━━━━━━━━━━━━ 18s 18ms/step - accuracy: 0.9145 - loss: 0.2687 - val_accuracy: 0.9713 - val_loss: 0.0886
Epoch 3/10
981/981 ━━━━━━━━━━━━━━━━━━━━ 19s 20ms/step - accuracy: 0.9404 - loss: 0.1911 - val_accuracy: 0.9805 - val_loss: 0.0604
Epoch 4/10
981/981 ━━━━━━━━━━━━━━━━━━━━ 20s 20ms/step - accuracy: 0.9562 - loss: 0.1451 - val_accuracy: 0.9772 - val_loss: 0.0670
Epoch 5/10
981/981 ━━━━━━━━━━━━━━━━━━━━ 18s 19ms/step - accuracy: 0.9633 - loss: 0.1200 - val_accuracy: 0.9236 - val_loss: 0.2531
Epoch 6/10
```

```
981/981 ━━━━━━━━━━━━━━━━━━━━ 20s 20ms/step - accuracy: 0.9635 - loss: 0.1161 - val_accuracy: 0.9841 - val_loss: 0.0519
Epoch 7/10
981/981 ━━━━━━━━━━━━━━━━━━━━ 19s 19ms/step - accuracy: 0.9665 - loss: 0.1093 - val_accuracy: 0.9904 - val_loss: 0.0305
Epoch 8/10
981/981 ━━━━━━━━━━━━━━━━━━━━ 19s 20ms/step - accuracy: 0.9757 - loss: 0.0773 - val_accuracy: 0.9744 - val_loss: 0.0955
Epoch 9/10
981/981 ━━━━━━━━━━━━━━━━━━━━ 22s 22ms/step - accuracy: 0.9720 - loss: 0.0936 - val_accuracy: 0.9874 - val_loss: 0.0374
Epoch 10/10
981/981 ━━━━━━━━━━━━━━━━━━━━ 21s 21ms/step - accuracy: 0.9761 - loss: 0.0778 - val_accuracy: 0.9834 - val_loss: 0.0477
Custom CNN training complete!
```

[22]:
```python
# Plot CNN Training History
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history_cnn.history['accuracy'], label='Train Accuracy')
plt.plot(history_cnn.history['val_accuracy'], label='Val Accuracy')
plt.legend()
plt.title('Custom CNN Accuracy (32x32)')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')

plt.subplot(1, 2, 2)
plt.plot(history_cnn.history['loss'], label='Train Loss')
plt.plot(history_cnn.history['val_loss'], label='Val Loss')
plt.legend()
plt.title('Custom CNN Loss (32x32)')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.tight_layout()
plt.show()
```



Step 6: Build and Train EfficientNetB0 (32x32) - OPTIMIZED FOR SMALL IMAGES

*[9]:
```python
# Build EfficientNetB0 Transfer Learning Model (32x32)
print("Building EfficientNetB0 model (32x32)...")
input_shape_eff = (IMG_HEIGHT, IMG_WIDTH, 3)
input_tensor_eff = Input(shape=input_shape_eff)

# Preprocess for EfficientNetB0
x = Lambda(preprocess_efficientnet)(input_tensor_eff)

# Base Model - EfficientNetB0 (lightweight, optimized for small inputs)
base_model_eff = EfficientNetB0(
    weights='imagenet',
    include_top=False,
    input_tensor=x,
    pooling='avg'
)
base_model_eff.trainable = False  # Freeze weights initially

# Custom classification head
x = base_model_eff.output
x = Dense(128, activation='relu')(x)
x = Dropout(0.5)(x)
predictions = Dense(num_classes, activation='softmax')(x)

model_eff = Model(inputs=input_tensor_eff, outputs=predictions)
model_eff.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
print("EfficientNetB0 model built (32x32).")
```

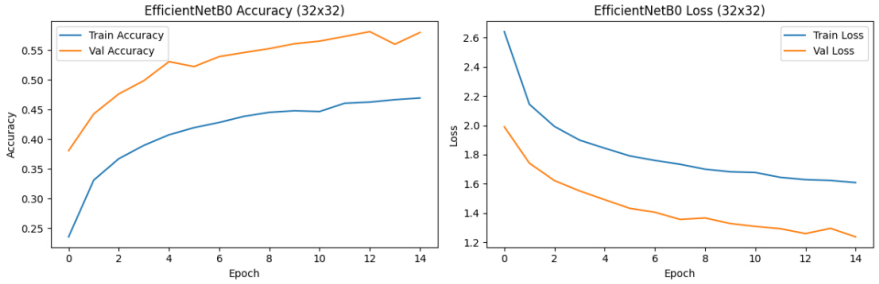
```
Building EfficientNetB0 model (32x32)...
WARNING:tensorflow:From C:\Users\yasser\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\site
-packages\keras\src\backend\tensorflow\core.py:232: The name tf.placeholder is deprecated. Please use tf.compat.v1.placeholder instead.

EfficientNetB0 model built (32x32).
```

[10]:
```python
# Train EfficientNetB0 (32x32)
print("\nTraining EfficientNetB0 (32x32)...")
datagen_eff = ImageDataGenerator(
    rotation_range=10,
    zoom_range=0.1,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=False
)
datagen_eff.fit(X_train_eff)

history_eff = model_eff.fit(
    datagen_eff.flow(X_train_eff, y_train_eff, batch_size=32),
    validation_data=(X_val_eff, y_val_eff),
    epochs=15,
    verbose=1
)
print("EfficientNetB0 training complete!")
```

```
Training EfficientNetB0 (32x32)...
Epoch 1/15
981/981 ━━━━━━━━━━━━━━━━━━━━ 44s 38ms/step - accuracy: 0.2356 - loss: 2.6406 - val_accuracy: 0.3808 - val_loss: 1.9902
Epoch 2/15
981/981 ━━━━━━━━━━━━━━━━━━━━ 36s 36ms/step - accuracy: 0.3311 - loss: 2.1438 - val_accuracy: 0.4426 - val_loss: 1.7415
Epoch 3/15
981/981 ━━━━━━━━━━━━━━━━━━━━ 33s 33ms/step - accuracy: 0.3673 - loss: 1.9921 - val_accuracy: 0.4764 - val_loss: 1.6222
Epoch 4/15
981/981 ━━━━━━━━━━━━━━━━━━━━ 32s 33ms/step - accuracy: 0.3897 - loss: 1.8989 - val_accuracy: 0.4987 - val_loss: 1.5520
Epoch 5/15
981/981 ━━━━━━━━━━━━━━━━━━━━ 32s 33ms/step - accuracy: 0.4075 - loss: 1.8438 - val_accuracy: 0.5307 - val_loss: 1.4919
Epoch 6/15
981/981 ━━━━━━━━━━━━━━━━━━━━ 32s 33ms/step - accuracy: 0.4195 - loss: 1.7911 - val_accuracy: 0.5224 - val_loss: 1.4326
Epoch 7/15
981/981 ━━━━━━━━━━━━━━━━━━━━ 32s 33ms/step - accuracy: 0.4283 - loss: 1.7601 - val_accuracy: 0.5393 - val_loss: 1.4062
Epoch 8/15
981/981 ━━━━━━━━━━━━━━━━━━━━ 33s 33ms/step - accuracy: 0.4387 - loss: 1.7340 - val_accuracy: 0.5462 - val_loss: 1.3570
Epoch 9/15
981/981 ━━━━━━━━━━━━━━━━━━━━ 32s 33ms/step - accuracy: 0.4454 - loss: 1.7004 - val_accuracy: 0.5527 - val_loss: 1.3670
Epoch 10/15
981/981 ━━━━━━━━━━━━━━━━━━━━ 32s 33ms/step - accuracy: 0.4480 - loss: 1.6825 - val_accuracy: 0.5610 - val_loss: 1.3291
Epoch 11/15
981/981 ━━━━━━━━━━━━━━━━━━━━ 32s 33ms/step - accuracy: 0.4467 - loss: 1.6778 - val_accuracy: 0.5653 - val_loss: 1.3093
Epoch 12/15
981/981 ━━━━━━━━━━━━━━━━━━━━ 32s 33ms/step - accuracy: 0.4606 - loss: 1.6446 - val_accuracy: 0.5732 - val_loss: 1.2935
Epoch 13/15
981/981 ━━━━━━━━━━━━━━━━━━━━ 32s 33ms/step - accuracy: 0.4626 - loss: 1.6290 - val_accuracy: 0.5814 - val_loss: 1.2604
Epoch 14/15
981/981 ━━━━━━━━━━━━━━━━━━━━ 32s 33ms/step - accuracy: 0.4667 - loss: 1.6232 - val_accuracy: 0.5601 - val_loss: 1.2958
Epoch 15/15
981/981 ━━━━━━━━━━━━━━━━━━━━ 32s 33ms/step - accuracy: 0.4695 - loss: 1.6088 - val_accuracy: 0.5800 - val_loss: 1.2378
EfficientNetB0 training complete!
```

[11]:
```python
# Plot EfficientNetB0 Training History
plt.figure(figsize=(12, 4))
```

```
plt.subplot(1, 2, 1)
plt.plot(history_eff.history['accuracy'], label='Train Accuracy')
plt.plot(history_eff.history['val_accuracy'], label='Val Accuracy')
plt.legend()
plt.title('EfficientNetB0 Accuracy (32x32)')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')

plt.subplot(1, 2, 2)
plt.plot(history_eff.history['loss'], label='Train Loss')
plt.plot(history_eff.history['val_loss'], label='Val Loss')
plt.legend()
plt.title('EfficientNetB0 Loss (32x32)')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.tight_layout()
plt.show()
```



Step 7: Load and Prepare Test Data (32x32)

[12]:
```
# Load test images (32x32)
test_data = []
test_labels = test_df['ClassId'].values

print("Loading test images (32x32)...")
for index, row in test_df.iterrows():
    img_path = os.path.join(DATASET_PATH, row['Path'])
    img = load_and_preprocess_image(img_path, target_size=(IMG_WIDTH, IMG_HEIGHT))
    if img is not None:
        test_data.append(img)
    if index % 5000 == 0:
        print(f"Processed {index} test images")

print(f"Loaded {len(test_data)} test images (32x32)")
```

```
Loading test images (32x32)...
Processed 0 test images
Processed 5000 test images
Processed 10000 test images
Loaded 12630 test images (32x32)
```

[13]:
```
# Prepare test data for both models (32x32)
X_test_raw = np.array(test_data)

# For CNN (normalized)
X_test_cnn = X_test_raw.astype('float32') / 255.0

# For EfficientNetB0 (with EfficientNet preprocessing)
X_test_eff = preprocess_efficientnet(X_test_raw.astype('float32'))

y_test = to_categorical(test_labels, num_classes)

print(f"Test CNN Data Shape (32x32): {X_test_cnn.shape}")
print(f"Test EfficientNetB0 Data Shape (32x32): {X_test_eff.shape}")
print(f"Test Labels Shape: {y_test.shape}")
```

```
Test CNN Data Shape (32x32): (12630, 32, 32, 3)
Test EfficientNetB0 Data Shape (32x32): (12630, 32, 32, 3)
Test Labels Shape: (12630, 43)
```

Step 8: Evaluate Both Models

[23]:
```
# Evaluate Custom CNN on test data
print("Evaluating Custom CNN (32x32)...")
loss_cnn, acc_cnn = model_cnn.evaluate(X_test_cnn, y_test, verbose=0)
print(f"Custom CNN Test Accuracy: {acc_cnn * 100:.2f}%")
print(f"Custom CNN Test Loss: {loss_cnn:.4f}")

# Evaluate EfficientNetB0 on test data
print("\nEvaluating EfficientNetB0 (32x32)...")
loss_eff, acc_eff = model_eff.evaluate(X_test_eff, y_test, verbose=0)
print(f"EfficientNetB0 Test Accuracy: {acc_eff * 100:.2f}%")
print(f"EfficientNetB0 Test Loss: {loss_eff:.4f}")
```

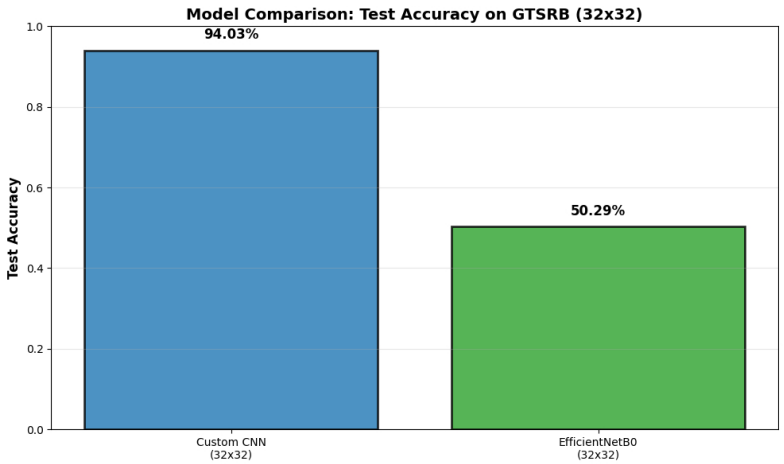
```
Evaluating Custom CNN (32x32)...
Custom CNN Test Accuracy: 94.03%
Custom CNN Test Loss: 0.2114

Evaluating EfficientNetB0 (32x32)...
EfficientNetB0 Test Accuracy: 50.29%
EfficientNetB0 Test Loss: 1.4874
```

Step 9: Model Comparison

[25]:
```
# Compare models
print("\n" + "="*70)
print("MODEL COMPARISON - ALL MODELS AT 32x32 (NATIVE GTSRB RESOLUTION)")
print("="*70)
print(f"Custom CNN (32x32):                {acc_cnn * 100:.2f}%")
print(f"EfficientNetB0 (32x32):            {acc_eff * 100:.2f}%")
print("="*70)
if acc_eff > acc_cnn:
    improvement = (acc_eff - acc_cnn) * 100
    print(f"\nEfficientNetB0 Improvement: +{improvement:.2f}%")
else:
    improvement = (acc_cnn - acc_eff) * 100
    print(f"\nCNN Better by: +{improvement:.2f}%")
print("="*70)

# Visualization
models_list = ['Custom CNN\n(32x32)', 'EfficientNetB0\n(32x32)']
accuracies = [acc_cnn, acc_eff]

plt.figure(figsize=(10, 6))
colors = ['#1f77b4', '#2ca02c']
bars = plt.bar(models_list, accuracies, color=colors, alpha=0.8, edgecolor='black', linewidth=2)
plt.ylim(0, 1.0)
plt.ylabel('Test Accuracy', fontsize=12, fontweight='bold')
plt.title('Model Comparison: Test Accuracy on GTSRB (32x32)', fontsize=14, fontweight='bold')
plt.grid(axis='y', alpha=0.3)

# Add value labels
for bar, acc in zip(bars, accuracies):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.02,
             f'{acc*100:.2f}%', ha='center', va='bottom', fontsize=12, fontweight='bold')

plt.tight_layout()
```

```
plt.show()
```

```
======================================================================
MODEL COMPARISON - ALL MODELS AT 32x32 (NATIVE GTSRB RESOLUTION)
======================================================================
Custom CNN (32x32):                94.03%
EfficientNetB0 (32x32):            50.29%
======================================================================

CNN Better by: +43.75%
======================================================================
```



Notes: Traffic signs are small, so we used 32x32. Upscaling images adds ugly interpolation artifacts without helping. We compared between two models, a simple custom CNN works great as a baseline, and EfficientNetB0 which is a lightweight transfer learning option that handles small images nicely. The accuracy of CNN custom model was better than EfficientNetB0.